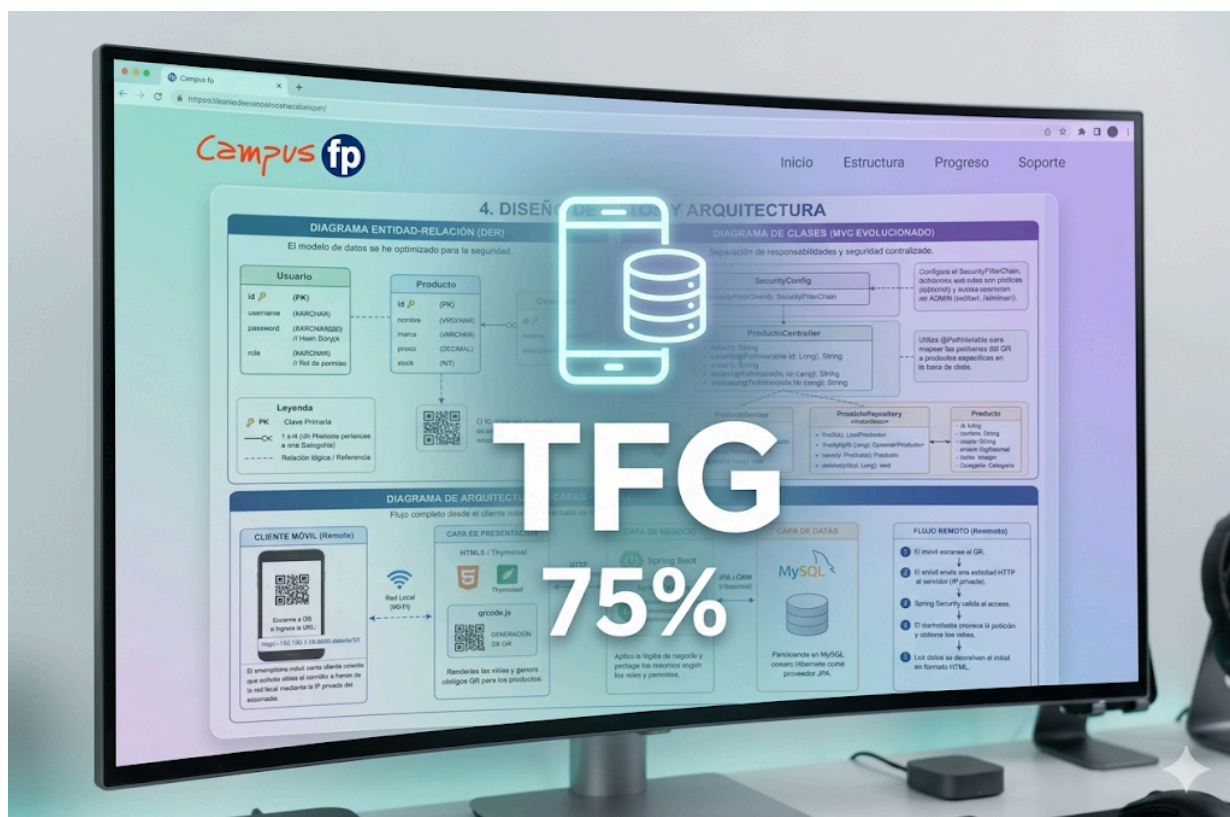




TÍTULO: Sistema de Gestión de Inventario para Tienda de Calzado Deportivo (Hito 3 - 75%)

AUTOR: Alejandro Tovar Barroso

TUTOR: David Valbuena Segura

CENTRO: CampusFP - Centro de Formación Profesional

FECHA: 5 de mayo de 2026

ÍNDICE

PORTADA.....	1
2. INTRODUCCIÓN CON MOTIVACIÓN Y OBJETIVOS.....	3
Motivación.....	3
Objetivos del Hito 3.....	3
Requisitos Funcionales (RF).....	4
Requisitos No Funcionales (RNF).....	4
4. DISEÑO DE DATOS Y ARQUITECTURA.....	4
Diagrama Entidad-Relación (DER).....	4
Diagrama de Clases (MVC Evolucionado).....	4
Diagrama de Arquitectura (3 Capas + Móvil).....	4
5. METODOLOGÍA Y PLANIFICACIÓN TEMPORAL.....	5
6. DESCRIPCIÓN TÉCNICA DEL DESARROLLO.....	5
7. MANUAL DE INSTALACIÓN Y USO.....	5
Instalación Técnica.....	5
Manual de Uso.....	5
8. CONCLUSIONES Y TRABAJO FUTURO.....	6
9. BIBLIOGRAFÍA.....	6

1. RESUMEN / ABSTRACT

Este documento presenta la evolución técnica del proyecto de gestión de inventario, alcanzando el **75% del desarrollo total**. La fase actual se ha centrado en la transición de una aplicación de gestión local a un ecosistema de red interoperable. Los pilares fundamentales de este hito han sido la fortificación de la seguridad mediante **cifrado BCrypt**, la orquestación de accesos basada en **Roles de Usuario (RBAC)** y la implementación de un sistema de **Generación de QR dinámicos**. Esta última funcionalidad permite la sincronización entre el stock físico y la base de datos digital mediante el uso de dispositivos móviles.

2. INTRODUCCIÓN CON MOTIVACIÓN Y OBJETIVOS

Motivación

La digitalización de pequeñas empresas de calzado suele enfrentar barreras de coste y complejidad. Este proyecto surge para ofrecer una solución robusta pero accesible, eliminando el error humano derivado de la gestión manual. La integración de la tecnología QR se justifica por la necesidad de movilidad de los operarios en el almacén.

Objetivos del Hito 3

- **Fortalecimiento Criptográfico:** Migrar de contraseñas en plano a un sistema de hashing irreversible con "salt" aleatorio.
- **Segmentación de Privilegios:** Implementar una lógica de negocio que separe las acciones de lectura de las de modificación crítica.
- **Conectividad Cross-Device:** Resolver los desafíos de red para que el servidor sea accesible desde IP privadas externas.
- **Automatización de Identificación:** Crear un flujo de generación de etiquetas QR sin intervención manual del usuario.

3. ANÁLISIS DE REQUISITOS

Requisitos Funcionales (RF)

- **RF6 - Gestión de Roles (RBAC):** El sistema debe identificar si un usuario posee el rol **ADMIN** (acceso total a CRUD) o **USER** (solo consulta de inventario y generación de QR).
- **RF7 - Seguridad de Credenciales:** Uso obligatorio de la interfaz **PasswordEncoder** para que las contraseñas en la base de datos sean ilegibles.
- **RF8 - Generación de QR Dinámico:** El sistema debe capturar el ID del producto y generar un código QR que contenga la URL absoluta de su ficha técnica.
- **RF9 - Public View (QR):** La ruta de detalle del producto debe ser accesible de forma anónima para facilitar el escaneo rápido desde dispositivos externos.

Requisitos No Funcionales (RNF)

- **RNF4 - Interoperabilidad de Red:** El servidor debe ser capaz de procesar peticiones procedentes de diferentes interfaces de red (LAN/WLAN).
- **RNF5 - Seguridad en Capa de Aplicación:** Implementación de un Firewall a nivel de software mediante Spring Security para interceptar peticiones no autorizadas.

4. DISEÑO DE DATOS Y ARQUITECTURA

Diagrama Entidad-Relación (DER)

El modelo de datos se ha optimizado para la seguridad:

- **Entidad Usuario:** Atributos **id**, **username**, **password** (almacena el hash BCrypt de 60 caracteres) y **role** (cadena que define el permiso).
- **Entidad Producto:** Atributos **id**, **nombre**, **marca**, **precio** y **stock**. Se relaciona con la lógica de QR mediante su **id** único.
- **Entidad Categoría:** Permite la agrupación lógica de productos (Nike, Adidas, etc.).

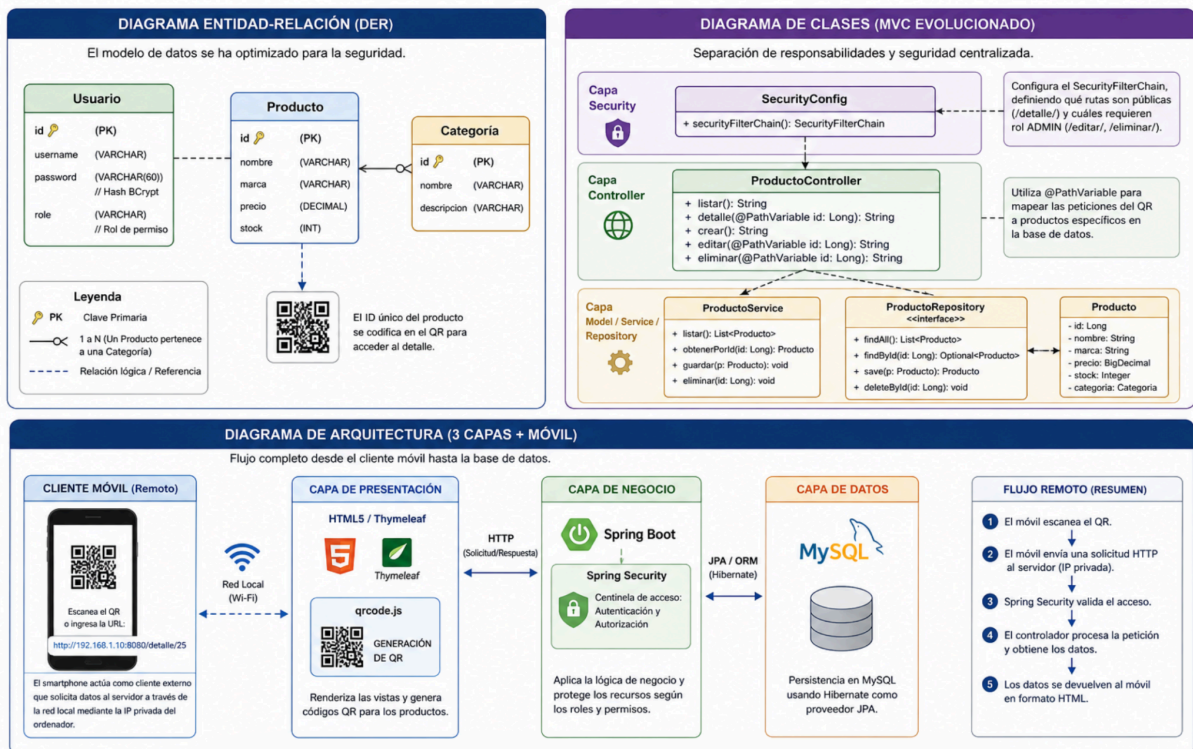
Diagrama de Clases (MVC Evolucionado)

- **Capa Security:** **SecurityConfig** configura el **SecurityFilterChain**, definiendo qué rutas son públicas (**/detalle/****) y cuáles requieren rol **ADMIN** (**/editar/****, **/eliminar/****).
- **Capa Controller:** **ProductoController** utiliza **@PathVariable** para mapear las peticiones del QR a productos específicos en la base de datos.

Diagrama de Arquitectura (3 Capas + Móvil)

1. **Capa de Presentación:** HTML5/Thymeleaf y la librería **qrcode.js**.
 2. **Capa de Negocio:** Spring Boot con Spring Security como centinela de acceso.
 3. **Capa de Datos:** Persistencia en MySQL con Hibernate.
- **Flujo Remoto:** El smartphone actúa como cliente externo que solicita datos al servidor a través de la red local mediante la IP privada del ordenador.

4. DISEÑO DE DATOS Y ARQUITECTURA



5. METODOLOGÍA Y PLANIFICACIÓN TEMPORAL

- Hito 1 (25%):** Definición de esquemas SQL y conexión JDBC inicial (Finalizado).
- Hito 2 (50%):** Desarrollo del Front-End responsivo y CRUD funcional (Finalizado).
- Hito 3 (75%): Actual Fase.** Integración de seguridad perimetral, gestión de roles y conectividad móvil vía QR (Finalizado).
- Hito 4 (100%):** Próxima fase. Reporting profesional en PDF y analítica de datos.

6. DESCRIPCIÓN TÉCNICA DEL DESARROLLO

- Implementación de BCrypt:** Se ha configurado un @Bean de BCryptPasswordEncoder. Este algoritmo es computacionalmente costoso y resistente a ataques de tablas arcoíris, lo que garantiza que, aunque la base de datos se vea comprometida, las contraseñas no puedan ser descifradas.
- Lógica de QR Dinámica:** En lugar de generar imágenes estáticas, se utiliza JavaScript para renderizar el QR en el cliente. El script concatena la IP privada del servidor con el endpoint del producto ([http://\[IP\]:8080/producto/detalle/\[ID\]](http://[IP]:8080/producto/detalle/[ID])), asegurando que el enlace sea válido para el móvil.
- Configuración de Red:** Se han habilitado reglas en el Firewall de Windows para permitir el tráfico entrante por el puerto 8080, permitiendo que el iPhone/Android del operario se comuniquen con el servidor Spring Boot.

7. MANUAL DE INSTALACIÓN Y USO

Instalación Técnica

1. **Entorno:** Importar proyecto Maven en IntelliJ IDEA.
2. **Red:** Ejecutar `ipconfig` en la consola para obtener la dirección IPv4 local.
3. **Configuración:** Editar la constante `url` en `inventario.html` con dicha IP para que los QR generados sean accesibles desde el móvil.

Manual de Uso

1. **Acceso:** El administrador se loguea para gestionar el inventario.
2. **Operativa:** Para identificar una zapatilla física, se pulsa el botón "QR". El sistema muestra el código en pantalla.
3. **Consulta:** Cualquier operario con el móvil escanea el código. Safari/Chrome abrirá la ficha técnica detallada sin necesidad de login previo, facilitando una consulta ágil.

8. CONCLUSIONES Y TRABAJO FUTURO

El proyecto ha validado la viabilidad de usar Spring Boot como una herramienta de gestión distribuida en red local. Se ha logrado un equilibrio entre **seguridad restrictiva** (para edición) y **accesibilidad abierta** (para consulta vía QR).

Próximos pasos:

- Generación de reportes de stock bajo en formato PDF mediante la librería **iText**.
- Incorporación de **Chart.js** para visualizar el balance de marcas en el inventario.

9. BIBLIOGRAFÍA

- Documentación de Seguridad de Spring Framework (docs.spring.io).
- Repositorio oficial de QRCode.js ([davidshimjs](https://github.com/davidshimjs)).
- Estándares de cifrado de contraseñas de la OWASP.